

The Hypothesis Graph: A Verifiable Semantic Memory for Coding Agents

Anonymous Author(s)

ABSTRACT

The *hypothesis graph* is a data structure for coding agents that deepens their reasoning and makes them accountable. Implemented at the harness layer, its nodes are testable claims, its edges the refutations that name the next claim. It updates by inquiry, Peirce’s typed loop of abduction, deduction, and induction. What is new is running that loop as mechanical harness operations around the one step no procedure reaches. That step, the abductive leap, stays in the model; the harness manufactures the surprise, fires the kill, and records the trail, instead of leaving the loop to a model’s undifferentiated prose. We demonstrate the mechanism on one contamination-free bugfix. Six self-attested arms plateau at a narrow fix; an externally verified comparator, the one input a context-bound agent cannot author for itself, carries a weaker model (Sonnet 4.6) to the behavior of the merged human fix that the strongest released model (Fable 5) does not reach without it. This buys verifiable accountability and reasoning that persists past the context window, at no additional training cost, usable by any coding-agent harness that can run pinned trials and read their verdicts.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation**; • **Computing methodologies** → *Artificial intelligence*.

KEYWORDS

hypothesis graph, agent memory, abductive inference, LLM agents, provenance, auditability, falsifiability, post-cutoff evaluation

ACM Reference Format:

Anonymous Author(s). 2026. The Hypothesis Graph: A Verifiable Semantic Memory for Coding Agents. In *Proceedings of the 1st International Workshop on Agentic Software Development (AgenticDev ’26)*. ACM, New York, NY, USA, 6 pages.

1 INTRODUCTION

In coding agents, the LLM is wrapped in a harness: the verification, testing, and memory a software task needs. The field building these is moving up a level of abstraction, from the model to the harness. Roychoudhury et al. [1] reframe the goal as *programming with trust*, arguing that deployment turns on verification, testing, and analysis built into the agent rather than on raw generation. Liu et al. [2] survey agents for software engineering and organize the field around those same missing pieces; Yehudai et al. [3] add that

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AgenticDev ’26, October 12, 2026, Munich, Germany

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

scoring final outputs misses the reasoning and failure causes inside a run, and call for trajectory-level assessment; Wang et al. [4] list persistent, structured memory among the open challenges.

A patch passes the visible tests, but that’s not enough: passing certifies only the cases the tests cover. An over-narrow patch passes them and is wrong off-suite. Confirming it means reconstructing the reasoning the agent never recorded, at a cost approaching that of producing it, so the work shifts from writing to checking. Code review is the bottleneck.

When reasoning is discarded, each run rebuilds context from scratch. Even where agent memory adopts the cognitive-architecture lineage, as CoALA [5] does in mapping it onto Soar [6] and ACT-R, the semantic slot stores facts rather than a falsifiable structure, so the search is discarded once a patch passes. At best, a trail of blobs is saved as provenance.

Here we give the agent a trail of verifiable knowledge. The **hypothesis graph** is a semantic memory that holds an agent’s reasoning while it works. The fix arrives with the inquiry that produced it: what was hypothesized, what was tested, what was ruled out. Each step carries a trial a stranger can rerun. The promise reverses the trust assumption by shifting the burden of proof outside the agent. The model still reasons, but each node it produces is checkable on its own at the harness layer, independent of the model.

Two contributions run through this, and they are separable. The graph is the accountability substrate, and its value is argued by construction: every node replays. The externalized comparator is the capability mechanism, and its value is shown by ablation. The graph makes reasoning checkable; the comparator supplies the one verdict a self-graded agent cannot author for itself. Keeping them apart is what lets the ablation below read cleanly, where several arms share the graph and only the verdict source moves.

What that buys is not incremental. On one contamination-free bug, an externalized comparator carried a weaker model to a fix the strongest released model could not reach without it (§5): a capability lift, where a scaffold usually buys only automation.

2 THE HYPOTHESIS GRAPH

2.1 Requirements

A semantic memory that holds an agent’s live reasoning must clear four requirements at once. *Holds hypotheses*: stores a hypothesis still under consideration, where prose, skill libraries, and retrieval keep only verified facts and established chunks. *Refutable by test*: each claim carries a kill condition, the executable test that can prove it wrong. *Independently verifiable*: a stranger verifies a conclusion by rerunning its recorded trial, instead of re-deriving the reasoning or taking the author’s word. *Persistent*: the trail persists past the context window rather than being discarded once a patch passes. Table 1 places prior structures against these requirements.

Table 1: Prior structures against the four requirements (• native, ◦ with common extensions, — needs another layer).

Structure	Hyp.	Tests	Indep. verif.	Persist.
Hypothesis graph	•	•	•	•
Truth-maintenance [7, 8]	•	◦	◦	◦
Provenance (W3C PROV [9])	◦	◦	◦	•
Search/proof tree [10, 11]	•	◦	◦	◦
Argumentation [12]	•	◦	◦	◦
ReAct trace [13]	•	◦	◦	•

An LLM is what finally fills the gap. Filling the graph takes a reasoner that reads a surprising failure, proposes candidate causes in open vocabulary, and turns each into an executable test, with no hand-built domain model. Classical inference engines could do this only inside a formalism encoded by hand, which is why the slot stayed a research program; the LLM populates it across arbitrary codebases, which is what makes the structure practical here.

2.2 Graph semantics

The structure meets all four requirements with three pieces: a node, an edge, and one invariant. A **node** is a claim bound to a trial: a hypothesis, the perturbation that tests it stated as an exact command, the observed outcome, and a credence typed by the reasoning mode that established it. An **edge** is generated by a kill condition: the manner of a hypothesis's death names the next hypothesis, so the structure is self-extending, with no external controller deciding where to look.

Replay invariant. Every node is reconstructible, by a stranger who does not trust the author, from its recorded trial alone: the exact command, the observed outcome, the verdict, and the credence cap.

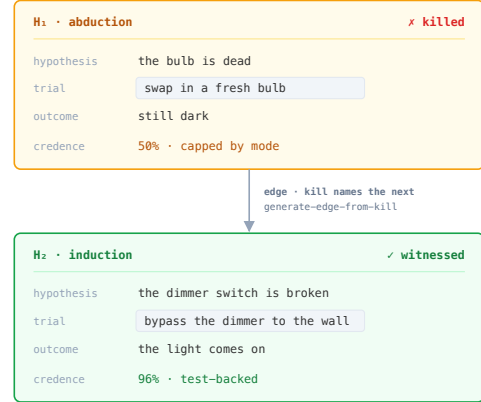
The contract holds on the node's mechanical skeleton; the hypothesis prose the node also carries falls outside it, by design. The prose is the part an auditor would otherwise have to trust, and the recorded trial is what replaces trusting it, so the invariant draws its line exactly where checkability begins.

Five operations maintain the structure, each defined with a one-clause argument that it preserves the invariant. *Create* appends a node; a node is admitted only if its trial replays, so the invariant holds by induction over reachable graphs. *Read/replay* reconstructs any node by re-running its recorded trial. *Classify* marks a node *killed* or *witnessed* on its trial's outcome and caps its credence at the mode that earned it; it appends a verdict and never edits the recorded trial. *Link* is the one genuinely novel operation: the manner of a hypothesis's death names the next hypothesis, so a classification spawns the next node with no external controller. *Prune* removes a dead branch from the working frontier while its record stays in place, so every pruned node replays exactly as before.

2.3 Auditability

The invariant yields a property no benchmark can evaluate:

Local replay auditability. Any single conclusion is checkable by re-running that node's one recorded



◦ replay - every node rebuilds from its recorded trial, by a reader who need not trust the author

Figure 1: Two nodes and the edge between them on a dead-light inquiry. The bulb hypothesis is killed by a cheap trial; its death names the next node, the dimmer, which a second trial witnesses. Each node binds a hypothesis to a trial, an observed outcome, and a credence capped by the mode that earned it. Every node rebuilds from its recorded trial, so an auditor replays the structure instead of trusting it.

trial, without reconstructing the inquiry and without extending trust to whoever ran it.

This is, for inquiry, the analogue of a certificate a consumer checks without trusting the producer, and like the Merkle audit path or proof-carrying code [14], its value rests on a contract; a complexity bound is beside the point. Two grades matter. Where the trial is a deterministic command over pinned inputs, replay is *strong*: re-execution reproduces the recorded outcome, and the lead case (§5) is here. Where the trial runs a model or a live service, replay is *artifact-level*: the recorded output is verified and the deterministic predicate re-run over it.

The nodes are ordinary; what is novel is the edge semantics. A search tree finds; a proof tree justifies. The hypothesis graph is both at once, because the search path is the justification: every step was a trial. Refinement is the closest kin: in CEGAR [10] and CEGIS [11] a counterexample is a kill that names the next experiment. What is novel is running that loop over an *open* hypothesis space abducted in domain vocabulary. Replayability stands in for the sound abstraction a closed setting supplies for free, with completeness as the price the open move forfeits. Truth-maintenance systems [7, 8] maintain belief status under assumptions, but the empirical trial and the kill-generated successor are external conventions; provenance [9] records replayable activities yet does not decide which hypothesis comes next; a ReAct trace [13] is an append-only log whose continuation policy the controller decides and the record never holds. What the hypothesis graph adds is their composition in one append-only object: an open-domain hypothesis, its executable trial, its kill condition, and the successor that kill names.

3 METHODEUTIC GROUNDING

A store of causal knowledge needs reasoning to be encoded into it. The insight, the leap to a candidate cause, stays with the model, but the harness needs a method to stage reasoning. Peirce [15, 16] types the operations of inquiry into three irreducible modes: *abduction* generates explanatory hypotheses for surprising observations; *deduction* derives testable predictions from hypotheses; *induction* tests predictions against evidence. No single mode carries a belief to its grade: abduction proposes content but does not test it; induction tests but introduces no new explanatory content; deduction traces consequences but invents nothing.

Collapse the modes and you get familiar failure modes: confirmation bias (induction without abductive alternatives), confabulation (abduction without inductive grounding), free-association (no typed mode at all). That collapse is exactly what modern LLM agents do by default, since a single forward pass proposes, predicts, evaluates, and rationalizes in undifferentiated prose.

The surprise has a primitive, and it is a diff: a before snapshot, an after snapshot, and the perturbation that flipped, read as figure against the ground that held. Separation logic calls the frame-inference half of this *bi-abduction* and scaled it in Facebook Infer [17]; it localizes where belief and code diverge and leaves the leap to what would explain it unmade. Debugging automation has run some version of this loop for decades: spectrum-based fault localization [18], delta debugging [19], model-based diagnosis [20], search-based repair [21, 22]. Those tools automate the loop; what is new is that the hypothesis graph *persists* it as a typed, replayable structure, and that an LLM abduces the hypothesis space open-vocabulary rather than inside a hand-built domain model. The difference from a Bayesian network [23] runs deeper than dropped probabilities: a Bayes net conditions over a fixed variable set it is handed; the hypothesis graph generates the space as the inquiry runs, its edges genealogical rather than conditional.

4 THE METHODEUTIC HARNESS

The loop that writes the graph needs four things: *iteration* (one pass can't be trusted, so the loop re-enters on failure); a *deterministic oracle* (a model can't be trusted to catch its own bias, so the check comes from outside the weights); *three typed modes*, each capped at its own confidence; and *semantic memory* (the reasoning is recorded into the hypothesis graph and survives the context window).

Concretely, this is a skill with a tool call in a loop: an outer deterministic driver invokes an agent via an *inquire* skill, which accesses a deterministic comparator tool (§5.4). *inquire* builds the graph with each node typed by the mode that established it: abduction proposes candidate root causes and writes hypothesis nodes with falsifiable predicates and kill conditions (low confidence); deduction traces each hypothesis's consequences to localize the suspect set (high); induction tests survivors with cheap read-only experiments (moderate). *implement* then writes the surviving hypothesis, with an adversarial challenger critiquing the diff against the spec. *at test* runs the test suite, takes the grader's pass/fail verdict, and emits a re-entry route from a fixed verdict-to-route table. The driver parses the verdict and the route; both are mechanical, and no model decides termination. Kill conditions are mechanical

predicates over the evidence trajectory, so a node dies when its predicate fires and not before. The graph persists across iterations; re-entry adds nodes rather than overwriting, so the graph doubles as the loop's checkpoint and no dead branch is re-proposed.

Code is the right substrate because it combines three properties other inquiry domains rarely bring together: it is *reproducible* (same input yields same output, modulo controlled nondeterminism), *deterministic* (causal lines from input to behavior are mechanical), and *perturbable* (single-line diffs are cheap to apply and fully observable). Because those three hold together, kill conditions over code are exact executions, and a single passing test on a captured diff is a complete verdict for that case.

5 THE MECHANISM BY CASE STUDY

What would prove that we had encoded reasoning into the harness, rather than just prompted a model into a better answer? The signal has to be one prompting alone can't produce: without the reasoning operation, the bare model can't reach the fix; handed the operation, it reaches the fix; and the fix generalizes to cases it was never shown.

5.1 Why not a benchmark

We first went looking for that signal on an established coding benchmark, and learned such benchmarks aren't built to show it. A SWE-bench-shaped task hands the solver the issue text, the repository, and a hidden test the fix must turn green; the issue text often hands over the specification, removing the discovery step. In our determinacy audit of all 728 public SWE-bench Pro [24] tasks (artifact: <https://anonymous.4open.science/r/weaudit-901E>), the well-specified majority is one-shot for a competent model, and the underdetermined floor (15.0%) grades the author's unstated intent, undiscoverable from the materials. With the tests as oracle, our harness resolves 95.3% of the public split under the official grader (artifact: <https://anonymous.4open.science/r/wepro-3D87>): an artifact of iterating against the answer key rather than a leaderboard result, and evidence that specification-to-implementation translation is largely solved. The interesting work is discovery, and the public set carries contamination risk besides. So we went looking for a post-cutoff bug where discovery is the whole difficulty.

5.2 Verus #2219

The bug is verus-lang/verus#2219: ghost uses of the never type invalidate borrow-checking. Verus is a deductive verifier for Rust; the cardinal property of any verifier is soundness, and an unsoundness, where the verifier blesses a program it owes a rejection, is the worst defect it can carry and the hardest to notice, since the symptom is silence. The issue is from March 2026, opened and fixed after the solve models' training cutoffs, so the case is contamination-free. The maintainer's narrow fix (PR #2230) passes its shipped test; a general fix landed three months later (PR #2501).

The correct fix must range over a whole class of inputs. A *ghost* expression of type `!`, erased before compilation and so not actually diverging, still triggers rustc's never-type edge prune: Verus drops a live control-flow edge and verifies a program it owes a rejection. Two programs identical at the `!` token owe opposite verdicts, depending on whether the divergence is ghost-erased or genuine.

Table 2: Ablation arms, least to most intervention (• present, ○ absent).

Arm	Enumeration	Kill cond.	Ext. verified
minimal / neutral prompt	○	○	○
site-enumeration prompt	•	○	○
abduction prompt	○	•	○
hypothesis-graph prompt	○	•	○
self-verifier harness	•	•	○
comparator-enabled harness	•	•	•

The narrow fix keys on the surface token; the general fix keeps the edge for the whole class of uninhabited types the token never names. That distinction, the symmetric difference (XOR) between the two behaviors, is what the experiment lives on. Three further properties make it a good study: the general fix took deep Verus knowledge and three months; the symptom sits four causal steps from the fault, so localization has to climb; and the project’s own suite passes for *both* fixes, so it cannot see the distinction.

5.3 Experiment design

We hold one reproducible control and ablate two factors for attribution (Table 2): the *verification mechanism*, whether each kill gets its verdict by the model’s own attestation (self-attested) or against a reference the model cannot author (externally verified); and the *inquiry tier*, whether the typed loop runs in the prompt or is lifted out as a tool call. Each arm writes a hypothesis graph by the same loop and runs three times, with one preregistered sentence per loop: testing X, predict Y, refuted by Z. For ecological accuracy each model runs in its own native agent CLI (Claude models in Claude Code, GPT-5.5 in codex, Composer 2.5 in Cursor); the meta-loop owns only the stage contracts, the graph, and the gate.

To score every arm on the same mechanical ruler we build a tiny answer key: three compiler states (the buggy base, the narrow fix, the general fix) plus a handful of probe programs, each owed one correct verdict, REJECT for unsound, ACCEPT for sound. Bug probes are uninhabited by construction, so their correct REJECT is free and the base compiler is wrong on exactly them. Divergence probes are the hard side: sound, to be preserved, and invisible to the gate’s grammar; their ACCEPT comes from the merged human fix and stays held out. Passing them therefore tests whether a fix represents the rule rather than fits the gate. A recall probe rules out memorization: asked in isolation how #2219 was fixed, the model does not recover the patch. Every verdict is from a forced-fresh, fingerprinted build; the frozen dataset and regrade script are committed (<https://anonymous.4open.science/r/hymech-117E>).

5.4 The comparator tool

The externalized comparator is released as an open-source command-line tool (<https://anonymous.4open.science/r/comparator-DA10>). It does three domain-general operations with no answer built in: it *enumerates* a case space closed under the property’s type-formers, wider than any one hypothesis; it *calibrates* each case against a known-good baseline, so the ground truth is external to the model; and it *gates* on a single pass/fail signal the model iterates against.

Table 3: Six self-attested arms vs. the comparator arm. Cases flipped is the modal of three draws.

Arm	Gate pass	Flipped	Outcome
minimal / neutral	—	114	narrow plateau
site-enumeration	—	114	narrow plateau
abduction	—	114	narrow plateau
hypothesis graph	—	114	narrow plateau
self-verifier	—	114	narrow plateau
comparator	✓	269	general on bug-set

Table 4: Two contamination-clean models, one harness, gate covering divergence. One draw per cell.

	No comparator	+ comparator
Table 5 (strongest released)	wide-but-broken	#2501 behavior
Sonnet 4.6	narrow (#2230)	#2501 behavior

The prompt demands generality and supplies the signal, so the rule is the model’s own reconstruction: the comparator returns only pass/fail on candidate behavior, never the patch, the predicate, or the API.

5.5 Results

The six self-attested methods plateaued at the narrow fix on all eighteen draws; only the externally verified arm broke past (Table 3). Its fix flips the whole bug-set, rejects two out-of-grammar held-outs it never saw, and stays wide-but-broken exactly where the gate’s grammar is silent, the divergence side. Extend the gate’s coverage to the divergence case and the same fix lands across four models in four native CLIs (Fable, Sonnet 4.6, Composer 2.5 reach the general fix’s behavior; codex clears the bug side but cannot implement the divergence case, an implementation limit the verdicts do not remove).

The headline lift is a slice across two models: the same bug handed to the strongest released model without the comparator, and to a weaker one with it (Table 4). Without the comparator neither matches the merged fix, and they miss in opposite directions: Fable too wide, Sonnet too narrow. With it both reach the merged fix’s behavior on the committed probes. The match is reconstruction, not lookup.

5.6 Why the cheaper explanations fall

Did the comparator just search harder? The self-verifier searched exactly as hard: handed the same prompt and no tool, it built its own case generator and brute-forced some 7,026 cases a round, and still plateaued, grading against its own belief. Enumeration sat on both sides of the comparison, so it is not the lever. Nor is reasoning depth: the abduction arm ran textbook bi-abduction to the semantic predicate the general fix turns on, named it in prose, self-validated it against its own programs, and still shipped the same narrow patch as the one-line minimal arm.

Was the model handed the fix? It was handed correct verdicts on cases. It never saw the patch, the predicate, or the Rust API; it still localizes the fault four steps up the chain and writes the fix itself.

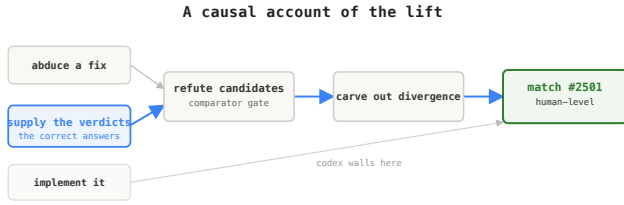


Figure 2: The ablation as one causal diagram. Four factors stay identical across arms (model, loop, bug, graph), so none can be the cause; only the verdict source varies. Gray boxes the model can build for itself; the blue box is the one input it cannot author.

A better scaffold, then? The self-verifier is the scaffold control: the whole loop, enumeration and kill conditions included, with the external verdict the only piece removed. It plateaus beside the bare prompt.

Just the stronger model? The attribution rests on the within-model contrast, where the model is held fixed and only the verdict source moves. The cross-model grid only illustrates the consequence; the within-model ablation proves the cause.

A model can bootstrap the *enumeration*, because that is mechanical: handed a vague prompt and no gate, Fable rebuilt one for itself, its own 7,026-case gate over the uninhabited type-formers. It cannot bootstrap the *correct verdicts*: its self-built gate labels each case with the very predicate under test, so the one case that needs a truth from outside its own belief gets self-mislabeled as handled. The claim is scoped to that self-grading circularity. Retrospectively the correct verdicts are free, mined from approved history (the merged fix, the regression suite); prospectively, on a fresh bug, the easy side has one and the hard side costs a human judgment. That division of labor is the deployment design, and nothing is hidden.

6 RELATED WORK

Scaffolds and harnesses. SWE-bench-targeted harnesses (OpenHands [25], SWE-agent [26], AutoCodeRover [27]) are ReAct-pattern loops without a falsifiable memory structure. Recent results report a weaker model with a scaffold beating a stronger one on the *automation* axis: GradleFixer [28] via domain-specific Gradle primitives, Confucius Code Agent [29] via a whole-scaffold effect that reverses when the stronger model gets the same scaffold. The lift here differs on three counts: it is a frontier extension rather than a throughput gain; the externalized operation is domain-general, carrying no answer and no domain model; and a controlled ablation pins it to the verdict source alone, so it does not reverse when the stronger model is handed the same harness. SWT-Bench [30] uses the same external-golden shape but as an evaluation oracle for test generation; ours is a runtime instrument the agent iterates against with the answer key hidden.

Typed reasoning and graph memory. Theorem-of-Thought [31] types reasoning into abductive/deductive/inductive agents per query: the typed cycle without a persistent typed memory. Cognitive Memory Manager [32] extracts a typed-node DAG by observing agent

execution: the typed graph, mined descriptively where ours is generative (kills fire live and route the run). IDEA [33] and ADI [34] run explicitly Peircean loops in non-SE domains. FVDebug [35] builds a debugging hypothesis graph but selects the next node by asking the model, the arbiter this work removes. What the ablation adds to this cluster is which piece is decisive: not the graph structure, which several share, but where a kill edge gets its ground truth.

Benchmark validity. SWE-Bench+ [36] found solution leakage and weak tests by manual audit; LiveCodeBench [37] originated post-cutoff evaluation. The standard objection, that cutoffs are porous, is answered here by construction: the case is post-cutoff and the solving model's weights predate the fix.

7 LIMITATIONS

The mechanism evidence is existence-grade: one audited divergence, on one instance, in a program that was not preregistered when it ran. Nothing here is a rate. On pilots from a merged-PR pool, a minimal baseline solved most bugs unaided; the band where the mechanism engages is bounded below by baseline reach and above by triage, which is why a population rate over an unselected pool would measure the band's width and miss the mechanism. The loop-versus-corpus control is unrun: whether the lift comes from iterating against the gate or from the labeled cases alone has a separating control (a one-shot pass over the static cases, no loop) that has not been run. The memory is small and per-instance, one mark-down file per inquiry; cross-instance accumulation is untested. The central claims are built to fail loudly: the mechanism claim dies if the committed receipt fails to replay or a discriminating program shows the externally verified fix wrong where the human fix is right; the encoding-boundary claim dies if a self-graded model reaches the hard side with labels it authored independently of the predicate under test; the attribution dies if the bracket fails to replicate from its preregistered sample.

8 CONCLUSION

In the domain where every step is checkable, an agent's reasoning can be made accountable rather than merely trusted. The hypothesis graph is a replayable record that submits each generated step to a world-facing trial and retains only what survives; it reduces the cost of relying on an agent from reproducing its work to rerunning its record. Inside that checkable domain sits a bolder result: on a single contamination-free bug, the externalized comparator carried Sonnet 4.6 to a fix Fable 5 could not reach without it, a capability lift attributed by within-model ablation to the verdict source, the one input the model cannot author, with no training involved.

Artifact availability and LLM disclosure

All code and data are openly available and DOI-archived; links are anonymized for review. The comparator tool, the mechanism experiment (preregistration, frozen dataset, regrade script, climb traces), the bench run, and the determinacy audit each reproduce from committed inputs. LLMs are the subject of study, an instrument, and a writing aid; claims, methodology, numbers, and argument structure are the authors'.

REFERENCES

- [1] A. Roychoudhury et al. 2025. AI Software Engineer: Programming with Trust. arXiv:2502.13767.
- [2] J. Liu et al. 2024. Large Language Model-Based Agents for Software Engineering: A Survey. arXiv:2409.02977.
- [3] A. Yehudai et al. 2025. Survey on Evaluation of LLM-based Agents. arXiv:2503.16416.
- [4] Y. Wang et al. 2025. A Survey of AI Agent Programming Systems. arXiv:2508.11126.
- [5] T. Summers, S. Yao, K. Narasimhan, and T. Griffiths. 2024. Cognitive Architectures for Language Agents. TMLR. arXiv:2309.02427.
- [6] J. Laird, A. Newell, and P. Rosenbloom. 1987. Soar: An Architecture for General Intelligence. *Artificial Intelligence* 33.
- [7] J. Doyle. 1979. A Truth Maintenance System. *Artificial Intelligence* 12.
- [8] J. de Kleer. 1986. An Assumption-based Truth Maintenance System. *Artificial Intelligence* 28.
- [9] L. Moreau et al. 2013. PROV-DM: The W3C Provenance Data Model.
- [10] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. 2000. Counterexample-Guided Abstraction Refinement. In *CAV*.
- [11] A. Solar-Lezama et al. 2006. Combinatorial Sketching for Finite Programs. In *ASPLOS*.
- [12] P. M. Dung. 1995. On the Acceptability of Arguments. *Artificial Intelligence* 77.
- [13] S. Yao et al. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*.
- [14] G. Necula. 1997. Proof-Carrying Code. In *POPL*.
- [15] C. S. Peirce. 1878. Illustrations of the Logic of Science. *Popular Science Monthly*.
- [16] C. S. Peirce. 1903. Pragmatism as the Logic of Abduction. Harvard Lectures on Pragmatism.
- [17] C. Calcagno, D. Distefano, P. O'Hearn, and H. Yang. 2009. Compositional Shape Analysis by Means of Bi-Abduction. In *POPL*.
- [18] J. Jones, M. J. Harrold, and J. Stasko. 2002. Visualization of Test Information to Assist Fault Localization. In *ICSE*.
- [19] A. Zeller and R. Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE TSE* 28.
- [20] R. Reiter. 1987. A Theory of Diagnosis from First Principles. *Artificial Intelligence* 32.
- [21] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE TSE* 38.
- [22] M. Monperrus. 2018. Automatic Software Repair: A Bibliography. *ACM Computing Surveys* 51.
- [23] J. Pearl. 1988. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann.
- [24] X. Deng et al. 2025. SWE-bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941.
- [25] X. Wang et al. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741.
- [26] J. Yang et al. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *NeurIPS*.
- [27] Y. Zhang et al. 2024. AutoCodeRover: Autonomous Program Improvement. In *ISSTA*.
- [28] GradleFixer. 2025. arXiv:2510.08640.
- [29] Confucius Code Agent. 2025. arXiv:2512.10398.
- [30] N. Mündler et al. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *NeurIPS*. arXiv:2406.12952.
- [31] S. Abdaljalil et al. 2025. Theorem-of-Thought: A Multi-Agent Framework for Abductive, Deductive, and Inductive Reasoning. arXiv:2506.07106.
- [32] Khalid and Arora. 2026. Cognitive Memory Manager. OpenReview yC-sHQnvvWY.
- [33] He et al. 2025. IDEA: Enhancing the Rule Learning Ability of Large Language Model Agents through Induction, Deduction, and Abduction. ACL Findings. arXiv:2408.10455.
- [34] S. Gilda and Gilda. 2026. ADI: A Peircean Tripartite Protocol over a Symbolic Knowledge Graph. arXiv:2604.15727.
- [35] FVDebug. 2025. arXiv:2510.15906.
- [36] R. Aleithan et al. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. arXiv:2410.06992.
- [37] N. Jain et al. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv:2403.07974.